Sistema de Gestión de Turnos Veterinarios

Documento de Relevamiento y Especificación — Etapa 1

1. Información del equipo

El equipo de trabajo está conformado por los siguientes integrantes:

- Junior Busso
- Gaspar Lynch
- Agustín Moreno
- Faustino Rodriguez
- Gonzalo Alvarez

2. Descripción del problema

Las veterinarias y clínicas para mascotas suelen gestionar sus turnos manualmente (por ejemplo, con agendas en papel o planillas), lo que puede generar horarios superpuestos entre profesionales, asignación de servicios a especialistas no habilitados para realizarlos, dificultad para saber la disponibilidad de un veterinario en el momento de agendar una consulta.

El objetivo de este proyecto es desarrollar, aplicando el paradigma de Programación Orientada a Objetos (POO) en Java, un sistema que digitalice y agilice el flujo de trabajo de una veterinaria, principalmente en la gestión de turnos. El sistema debe garantizar la coherencia entre clientes, mascotas, profesionales y servicios, validando las reglas de negocio (compatibilidad de especialidad, disponibilidad horaria, ventanas de anticipación y cancelación) para evitar errores de agenda y mejorar la experiencia tanto del cliente como del personal de la veterinaria.

En esta primera etapa se definen las reglas de negocio, se diseñan preliminarmente las clases y se delimitan las funcionalidades que se implementarán y las que no.

3. Reglas de negocio

3.1 Clientes

- Un mismo cliente puede tener registradas varias mascotas, pero cada turno se agenda de forma individual por mascota, identificado con un ID propio.

3.2 Profesionales

- Un veterinario o especialista no puede tener dos turnos asignados en el mismo horario.
- Determinados servicios solo pueden ser asignados a profesionales con la especialidad requerida para ese servicio.

3.3 Turnos

- Cada turno debe estar asociado a una única mascota

4. Glosario de términos

Cliente: Persona responsable de una o más mascotas, encargada de solicitar y gestionar los turnos.

Mascota: Animal registrado en el sistema bajo la responsabilidad de un cliente (dueño), sobre el cual se agendan los turnos.

Profesional: Veterinario o especialista habilitado para atender turnos, identificado por su matrícula y especialidad.

Especialidad: Área de competencia de un profesional (por ejemplo: clínica general, cirugía, dermatología) que determina qué servicios puede prestar.

Servicio: Prestación ofrecida por la veterinaria (consulta, cirugía, vacunación, etc.), que puede requerir una o más especialidades específicas.

Turno: Reserva de un horario determinado que vincula a una mascota, un profesional y un servicio.

Disponibilidad horaria: Conjunto de horarios en los que un profesional o recurso (como el quirófano) se encuentra libre para ser asignado a un turno.

Cancelación: Acción de anular un turno previamente agendado, sujeta al límite de tiempo mínimo definido en las reglas de negocio.

5. Bocetos preliminares de las clases

A continuación, se presenta el diseño preliminar de las clases con sus atributos principales. Este boceto no necesariamente representa la estructura final de las clases.

Clase: Cliente

Atributo	Tipo	Descripción
id	int	Identificador único del cliente
nombre	String	Nombre del cliente
apellido	String	Apellido del cliente
telefono	String	Número telefónico del cliente
mascotas	List<Mascota>	Mascotas del cliente

Clase: Mascota

Atributo	Tipo	Descripción
id	int	Identificador único de la mascota
animal	String	Especie o tipo de animal (perro, gato, etc.)
nombre	String	Nombre de la mascota
dueño	Cliente	Cliente registrado como dueño de la mascota

Clase: Profesional

Atributo	Tipo	Descripción
matricula	int	Número de matrícula, identificador único del profesional
nombre	String	Nombre del profesional
apellido	String	Apellido del profesional
telefono	String	Número telefónico del profesional
correo	String	Correo electrónico de contacto del profesional
horarios	List<>	Horarios de disponibilidad del profesional
especialidad	String	Especialidad del profesional

Clase: Servicio

Atributo	Tipo	Descripción
id	int	Identificador único del servicio
nombre	String	Nombre del servicio (consulta, cirugía, vacunación, etc)
especialidadRequerida	List<String>	Especialidades habilitadas para prestar este servicio

Clase: Turno

Atributo	Tipo	Descripción
id	int	Identificador único del turno
profesional	Profesional	Profesional asignado al turno
mascota	Mascota	Mascota para la cual se solicita el turno
horario	Time	Fecha y hora del turno
servicio	Servicio	Servicio solicitado
cancelado	boolean	Indica si el turno fue cancelado

6. Requerimientos funcionales que el equipo desarrollará

- Pedir turno: permitir a un cliente solicitar un turno para una mascota, validando especialidad, disponibilidad y ventana de anticipación.
- Cancelar turno: permitir a un cliente o a un profesional cancelar un turno existente.
- Ver disponibilidad horaria de un profesional: consultar los horarios libres de un profesional dentro del horario de atención de la veterinaria.

7. Requerimientos funcionales que no serán desarrollados en el cuatrimestre

- Atender una mascota (registro clínico de la consulta, diagnóstico, tratamiento, etc.).
- Vender mascotas.

SPRINT 1

Relaciones entre objetos

El modelo está compuesto por cinco clases de dominio: Cliente, Mascota, Profesional, Servicio y Turno. La clase Turno es el centro del modelo: es la que vincula a una mascota con el profesional que la atiende y el servicio que se presta.

Relación	Tipo	Multiplicidad	Navegabilidad	Implementación
Cliente – Mascota	Agregación	1 cliente → 0..* mascotas	Cliente → Mascota	List<Mascota> mascotas en Cliente
Turno – Mascota	Asociación	0..* turnos → 1 mascota	Turno → Mascota	atributo Mascota mascota en Turno
Turno – Profesional	Asociación	0..* turnos → 1 profesional	Turno → Profesional	atributo Profesional profesional en Turno
Turno – Servicio	Asociación	0..* turnos → 1 servicio	Turno → Servicio	atributo Servicio servicio en Turno
Profesional – Servicio	Relación lógica (sin referencia directa)	—	—	se comparan especialidad y especialidadRequerida

1. Cliente – Mascota (agregación, 1 a muchos)

Un cliente puede tener registradas cero o más mascotas. Para eso, Cliente guarda una lista **`List<Mascota>`**, que se inicializa vacía en el constructor y se completa con el método **`agregarMascota(Mascota)`** . La consideramos una agregación y no una composición porque la mascota se crea fuera del cliente y se le pasa ya construida, así que su ciclo de vida no depende del objeto Cliente.
La relación es unidireccional: el cliente conoce a sus mascotas, pero la mascota no guarda una referencia a su dueño. Tomamos esta decisión para mantener simple a la clase Mascota y evitar tener que sincronizar las dos referencias cada vez que se agrega una mascota. Para saber a qué cliente pertenece una mascota, se recorren las listas de mascotas de los clientes.

2. Turno – Mascota (asociación, muchos a 1)

Cada turno está asociado a una única mascota, como pide la regla de negocio 3.3. A su vez, una misma mascota puede tener varios turnos a lo largo del tiempo. Turno guarda la referencia a la mascota y no al cliente porque el turno se agenda por mascota, no por persona. La mascota no conoce sus turnos, así que la relación es unidireccional.

3. Turno – Profesional (asociación, muchos a 1)

Cada turno tiene asignado exactamente un profesional, y un profesional puede atender muchos turnos. El profesional no guarda la lista de sus turnos: es el turno el que conoce al profesional. Esta relación es la base para validar la regla 3.2: un profesional no puede tener dos turnos en el mismo horario. Para verificarlo hay que recorrer los turnos existentes y comparar el profesional, la fecha y el horario de cada uno.

4. Turno – Servicio (asociación, muchos a 1)

Cada turno corresponde a un único servicio (consulta, vacunación, cirugía, etc.), y un mismo servicio puede aparecer en muchos turnos. El servicio se modela como un objeto independiente y no como un texto dentro del turno. Así sus datos, como las especialidades requeridas, se definen una sola vez y los comparten todos los turnos que lo usan.

5. Profesional – Servicio (relación lógica a través de la especialidad)

No hay una referencia directa entre estas dos clases. La compatibilidad se determina comparando el atributo **`especialidad`** del Profesional con la lista **`especialidadRequerida`** del Servicio: un profesional puede prestar un servicio solo si su especialidad está incluida en esa lista. La decisión de que ninguna de las dos clases conozca a la otra evita acoplarlas, y la validación se hace en el momento de crear el turno.

Turno como clase asociativa

Turno funciona como una clase asociativa entre Mascota, Profesional y Servicio. No existe un vínculo directo entre esas tres clases: la relación entre ellas surge únicamente cuando se agenda un turno, y el turno además agrega información propia de ese encuentro (fecha, horario y si fue cancelado).

Decisiones de diseño

Acá explicamos qué decidimos al pasar del boceto del Sprint 0 al código, y por qué.

Qué cambió a comparacion del Sprint 0

cambiamos algunas cosas del boceto:

Clase	Antes (Sprint 0)	Ahora (código)	Por qué
Mascota	animal	especie y raza	"Especie" es la palabra correcta y la raza le sirve al veterinario
Mascota	dueño	Se sacó	El cliente ya sabe cuáles son sus mascotas
Profesional	horarios	Todavía no está	Lo vamos a agregar cuando programemos la disponibilidad horaria
Turno	horario	fecha y horario	Es más fácil buscar los turnos de un día

1. Los datos de cada clase están protegidos

Todos los atributos son private. Desde afuera solo se pueden leer con los métodos get. No hay métodos set para cambiarlos.
Lo hicimos así para que nadie cambie un dato por error. Por ejemplo: si cualquiera pudiera cambiar el profesional de un turno, se podría poner a alguien que no tiene la especialidad necesaria. Cuando algo sí tiene que cambiar, usamos un método con un nombre claro, como agregarMascota() en Cliente.

2. Cada objeto tiene su número (id)

Cliente, Mascota, Servicio y Turno tienen un id. Sirve para no confundir dos cosas que se llaman igual, por ejemplo dos perros llamados "Firulais".
El profesional no tiene id porque usa su matricula, que ya es un número único de cada veterinario.

3. Mascota

Cambiamos animal por especie (perro, gato, etc.), que es la palabra correcta, y agregamos raza porque es un dato que el veterinario necesita.
Sacamos el atributo dueño. El cliente ya tiene la lista de sus mascotas, así que guardar el dueño también en la mascota sería repetir el mismo dato dos veces.

4. Profesional

Cada profesional tiene una sola especialidad. Es lo más simple y alcanza para una veterinaria normal.

5. Servicio

Un servicio guarda una lista de especialidades y no una sola, porque algunos servicios los pueden hacer profesionales de distintas especialidades. Por ejempl: poner una vacuna la puede poner un veterinario clínico o un especialista.
Un profesional puede hacer un servicio solo si su especialidad está en esa lista.

6. Las especialidades se escriben como texto

Por ahora las especialidades son texto.
El problema es que hay que escribirlas siempre igual, por ejemplo cuando escribis "Cirugía" y "Cirugia" el programa las toma como distintas. Más adelante podríamos usar un enum, que de solo opciones.

7. Turno: fecha y hora separadas

En el boceto el turno tenía un solo dato horario. Ahora tiene fecha y horario separados. Así es más fácil hacer dos cosas que necesitamos: ver si un profesional ya tiene un turno ese día a esa hora, y ver qué horarios le quedan libres en un día.

8. Los turnos cancelados no se borran

Cuando se cancela un turno no lo borramos, solo lo marcamos con cancelado. Así queda registrado que existió, pero el horario vuelve a quedar libre para otra mascota.

9. El turno guarda los objetos completos

El turno guarda la mascota, el profesional y el servicio completos, no solo sus números de id. Así, desde el turno se puede ver directamente la especialidad del profesional (turno.getProfesional().getEspecialidad()) sin tener que buscarlo en una lista.

Responsabilidades de las clases

Siguiendo el principio de Responsabilidad Única (SRP - Single Responsibility Principle) de la programación orientada a objetos, cada clase del sistema asume un conjunto delimitado y coherente de responsabilidades dentro del modelo de dominio:

1. Cliente

- **Representación del titular:** Modela a la persona responsable y titular de contacto ante la clínica veterinaria (dueño de una o más mascotas).
- **Gestión de identidad y contacto:** Almacena y encapsula sus atributos personales (id, nombre, apellido, teléfono).
- **Administración de mascotas:** Mantiene la colección de mascotas a su cargo (List<Mascota>) y proporciona el método agregarMascota(Mascota) para incorporar nuevos animales sin delegar esta lógica a agentes externos.

2. Mascota

- **Representación del paciente:** Modela al animal que recibe los cuidados médicos y servicios dentro de la veterinaria.
- **Encapsulamiento biológico:** Resguarda la identidad y características del animal (id, nombre, especie, raza).
- **Entidad individual de turnos:** Funciona como el sujeto central e individual sobre el cual se agendan los turnos, respetando la regla de negocio de que cada turno corresponde a una única mascota.

3. Profesional

- **Representación del personal médico:** Modela al veterinario o especialista habilitado para prestar los servicios de la clínica.
- **Identificación y especialidad:** Encapsula su número de matrícula profesional, sus datos de contacto (teléfono, correo) y su especialidad médica correspondiente.
- **Validación de atención y disponibilidad:** Provee la información esencial para contrastar su especialidad frente a los servicios solicitados y controlar que no existan superposiciones horarias en su agenda.

4. Servicio

- **Tipificación de prestaciones:** Modela el catálogo de servicios ofrecidos por la veterinaria (consulta general, cirugía, vacunación, etc.).
- **Requerimientos de habilitación:** Encapsula la lista de especialidades requeridas (List<String> especialidadRequerida) necesarias para prestar la atención.
- **Desacoplamiento de reglas:** Permite desacoplar el servicio de los profesionales específicos, funcionando como criterio de validación al momento de asignar un turno.

5. Turno

- **Objeto asociativo central:** Materializa la reserva concreta de un horario determinado, vinculando a una Mascota, un Profesional y un Servicio solicitado.
- **Control de datos temporales:** Registra y preserva la fecha y hora de la cita veterinaria.
- **Gestión de ciclo de vida y estado:** Controla si el turno se encuentra activo o cancelado (atributo boolean cancelado), permitiendo gestionar cancelaciones según los límites de tiempo de la veterinaria.

Cliente.java

```java
import java.util.ArrayList;
import java.util.List;

public class Cliente {
    private int id;
    private String nombre;
    private String apellido;
    private String telefono;
    private List<Mascota> mascotas;

    public Cliente(int id, String nombre, String apellido, String telefono) {
        this.id = id;
        this.nombre = nombre;
        this.apellido = apellido;
        this.telefono = telefono;
        this.mascotas = new ArrayList<>();
    }

    public void agregarMascota(Mascota mascota) {
        mascotas.add(mascota);
    }

    public int getId() {
        return id;
    }

    public String getNombre() {
        return nombre;
    }

    public String getApellido() {
        return apellido;
    }

    public String getTelefono() {
        return telefono;
    }

    public List<Mascota> getMascotas() {
        return mascotas;
    }
}
```

Mascota.java

```java
public class Mascota {
    private int id;
    private String nombre;
    private String especie;
    private String raza;

    public Mascota(int id, String nombre, String especie, String raza) {
        this.id = id;
        this.nombre = nombre;
        this.especie = especie;
        this.raza = raza;
    }

    public int getId() {
        return id;
    }

    public String getNombre() {
        return nombre;
    }

    public String getEspecie() {
        return especie;
    }

    public String getRaza() {
        return raza;
    }
}
```

Profesional.java

```java
public class Profesional {
    private int matricula;
    private String nombre;
    private String apellido;
    private String telefono;
    private String correo;
    private String especialidad;

    public Profesional(int matricula, String nombre, String apellido, String
telefono, String correo, String especialidad) {
        this.matricula = matricula;
        this.nombre = nombre;
        this.apellido = apellido;
        this.telefono = telefono;
        this.correo = correo;
        this.especialidad = especialidad;
    }

    public int getMatricula() {
        return matricula;
    }

    public String getNombre() {
        return nombre;
    }

    public String getApellido() {
        return apellido;
    }

    public String getTelefono() {
        return telefono;
    }

    public String getCorreo() {
        return correo;
    }

    public String getEspecialidad() {
        return especialidad;
    }
}
```

Servicio.java

```java
import java.util.ArrayList;
import java.util.List;

public class Servicio {
    private int id;
    private String nombre;
    private List<String> especialidadRequerida;

    public Servicio(int id, String nombre) {
        this.id = id;
        this.nombre = nombre;
        this.especialidadRequerida = new ArrayList<>();
    }

    public int getId() {
        return id;
    }

    public String getNombre() {
        return nombre;
    }

    public List<String> getEspecialidadRequerida() {
        return especialidadRequerida;
    }
}
```

Turno.java

```java
import java.sql.Time;
import java.util.Date;

public class Turno {
    private int id;
    private Mascota mascota;
    private Profesional profesional;
    private Servicio servicio;
    private Date fecha;
    private Time horario;
    private boolean cancelado;

    public Turno(int id, Mascota mascota, Profesional profesional, Servicio
servicio, Date fecha, Time horario, boolean cancelado) {
        this.id = id;
        this.mascota = mascota;
        this.profesional = profesional;
        this.servicio = servicio;
        this.fecha = fecha;
        this.horario = horario;
        this.cancelado = cancelado;
    }

    public int getId() {
        return id;
    }

    public Mascota getMascota() {
        return mascota;
    }

    public Profesional getProfesional() {
        return profesional;
    }

    public Servicio getServicio() {
        return servicio;
    }

    public Date getFecha() {
        return fecha;
    }

    public Time getHorario() {
        return horario;
    }

    public boolean isCancelado() {
        return cancelado;
```

```
    }
}
```

Main.java

```
import java.sql.Time;
import java.util.Date;

public class Main {
    public static void main(String[] args) {
        // Crear instancias de Mascota, Profesional y Servicio
        Mascota mascota = new Mascota(1, "Firulais", "Perro", "Labrador");
        Profesional profesional = new Profesional(12345, "Juan", "Pérez",
"123456789", "juan.perez@email.com", "Veterinario");
        Servicio servicio = new Servicio(1, "Consulta General");
        Turno turno = new Turno(1, mascota, profesional, servicio, new Date(), new
Time(System.currentTimeMillis()), false);
    }
}
```